

Символы и строки

В языке программирования C++

Строковые переменные

- В С отсутствуют встроенные строковые типы в том смысле, в котором они есть в языках типа Basic и Pascal. Поэтому присущая этим языкам легкость оперирования строковыми переменными (присвоение, сравнение) в С++ недоступна.
- Строка – это **последовательность символов**. Последовательности в С++ представляются массивами или указателями.

- **Символ** – элементарная единица, некоторый набор которых несет определенный смысл.
- В языке программирования C++ предусмотрено использование символьных констант.
- **Символьная константа** – это целочисленное значение (типа `int`) представленное в виде символа, заключённого в одинарные кавычки, например `'a'`.
- В таблице ASCII представлены символы и их целочисленные значения.

ASCII (American standard code for information interchange)

Таблица расширенного кода ASCII.

Номер символа		Д Е С Я Т К И																											
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25		
ЕДИНИЦЫ	0	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	1	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	2	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	3	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	4	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	5	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩
	6	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	7	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	8	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	
	9	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩
10	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
11	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
12	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
13	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
14	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
15	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
16	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
17	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
18	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
19	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
20	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
21	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
22	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
23	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
24	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	
25	☺	☹	♥	♦	♣	♠	•	◼	♂	♀	↕	↗	↘	↖	↙	↚	↛	↜	↝	↞	↠	↡	↢	↣	↤	↥	↦	↩	

Escape-последовательности СИМВОЛОВ

- Сочетания символов, состоящих из косой черты (\), за которой следует буква или набор цифр, называются **escape-последовательностями**.
- Escape-последовательность рассматривается как один символ и, следовательно, является допустимой символьной константой.
- Escape-последовательности обычно используются для указания действий, например, возврата каретки или табуляции, на терминалах и принтерах. Они также используются для предоставления буквенных представлений непечатаемых символов, а также символов, которые обычно имеют специальное значение, например двойных кавычек (").

char

```
char newline = '\n';  
char tab = '\t';  
char backspace = '\b';  
char backslash = '\\';  
char nullChar = '\0';  
char symbol = 'h';
```

Значение	Escape-последовательность
новая строка	\n
горизонтальная табуляция	\t
вертикальная табуляция	\v
Удаление предыдущего символа	\b
возврат каретки	\r
Подача страницы (для перехода к началу следующей страницы)	\f
Звуковой сигнал (звонок)	\a
обратная косая черта	\\
вопросительный знак	? или \?
одинарная кавычка	\'
двойная кавычка	\"
нуль-символ	\0
восьмеричный	\ooo
шестнадцатеричный	\xhhh
Юникод (UTF-8)	\uxxxx
Юникод (UTF-16)	\Uxxxxxxxx

Изменение регистра символа

- `int tolower(int c);` // преобразование символа - латинской буквы верхнего регистра к нижнему, остальные символы не меняются
- `int toupper(int c);` // преобразование латинской буквы нижнего регистра к верхнему
- Пример:
`char c = 'E', t = tolower(c);`
`char c1 = 'e', t 1= toupper(c1);`

Проверка типа символа

- возвращают 0, если символ не удовлетворяет условию
- `int isdigit(int c);` // десятичная цифра (0-9)
- `int isalpha(int c);` // латинская буква (A-Z,a-z)
- `int isspace(int c);` // символ - эквивалент пробела (пробел, табуляция, перевод строки, перевод страницы, возврат каретки)
- `int isalnum(int c);` // латинская буква или цифры (A-Z,a-z,0-9)
- `int ispunct(int c);` // символ пунктуации
- `int isgraph(int c);` // видимый (отображаемый) символ
- `int isupper(int c);` // символ верхнего регистра клавиатуры
- `int islower(int c);` // символ нижнего регистра клавиатуры

Классификация строк в C++

- **C-строки** –
различные
вариации `char*`
- **C++-строки** –
вариации `std::basic_string<...>`

C-строка

- Строка, которая завершается символом с нулевым кодом – `'\0'`.
- `\0` – символ нуль-терминатора.
- Задается с помощью команд:
`char[]`
`char*`

Массив символов `char[]`

- Используется для *строк постоянной длины*.
- Должен иметь размер хотя бы на единицу больше фактического размера строки для хранения `\0`.

Пример объявления массива символов:

```
char strM1[] = "Hello, World!";  
char strM2[20] = "Yes";
```

Массив `strM1` состоит из 14 символов, массив `strM2` - из 20, но его четвертый символ имеет код `0`, сигнализирующий о фактическом конце строки.

Примеры C-строк

```
//инициализация строки из 10 символов, на последнем месте '\0'  
char str1[10] = "abcdefghf";
```

```
//посимвольная инициализация строки:  
char str2[10] =  
{ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'f', '\0' };  
//инициализация строки без указания размера  
char str3[] = "abcdefghf";
```

Стандартный ввод-вывод С-строк

```
char st [500]; // строковый массив
scanf_s ("%s", &s, _countof(s) );
// считываются введённые символы до первого пробела,
// для функции _countof(s) необходимо подключить stdlib.h
printf_s ("Print %s ", s); // вывод строки
#include <stdio>
gets_s (st); // считываются все введённые символы с пробелами
// до тех пор, пока не будет нажата клавиша Enter

puts (str); // записывает в поток содержимое строки
```

ПОТОКОВЫЙ ВВОД-ВЫВОД С-строк

```
#include <iostream>
```

```
cin >> st; //из всего введённого считается  
// последовательность символов до первого пробела
```

```
cin.getline(st, _countof(st)); // считываются все введённые  
// символы с пробелами до тех пор,  
// пока не будет нажата клавиша Enter
```

```
cout << st; // вывод строки
```

Доступ к символам внутри строки

Получение и изменение символа:

```
char str[] = "abcdefghf";
```

```
str[2] = '2'; //str = "ab2defghf"
```

```
*(str + 3) = '3'; //str = "ab23efghf"
```

```
char *p = str; //указатель на начало строки p=&str[0];
```

```
*p = '0'; //str = "0b23efghf"
```

```
++p;
```

```
*p = '1'; //str = "0123efghf"
```

Перебор символов строки

```
char str[] = "abcdefghf";  
for (int i = 0; i < strlen(str); ++i)  
    str[i] = toupper(str[i]); // "ABCDEFGHF"
```

strlen(const char*) - возвращает длину строки (без нулевого символа конца строки);

```
for (char* p = str; *p; ++p)  
    *p = tolower(*p); // "abcdefghf"
```

```
char* p = str;  
while (*p) {  
    *p++ = toupper(*p);  
} // "ABCDEFGHF"
```


Функции копирования

- **strcpy_s(char*, const char*)** - копирует одну строку в другую;
- **strncpy_s(char*, const char*, size_t)** - копирует n первых символов одной строки в другую;
- **strcat_s(char*, const char*)** - добавляет одну строку к другой;
- **strncat_s(char*, const char*, size_t)** - добавляет n символов одной строки к другой.

```
char str[] = "abcdefghf", t[30];  
strcpy_s(t, str), puts(t);  
strncpy_s(t, str, 5), puts(t);  
strcat_s(t, str), puts(t);  
strncat_s(t, str, 6), puts(t);
```

ФУНКЦИИ

- **strtok_s(char*, const char*, char**)** - возвращает следующий токен (элемент разбора).

```
char str[] = "ab c,de fg.hf", t[] = " ,.";
char *contex = 0;
char *p = strtok_s(str, t, & contex );
while (p) {
    cout << p<<" - ";
    p = strtok_s(0, t, & contex );
}
```

Функции поиска и замены

- **strchr(const char*, int)** - ищет первое вхождение указанного символа в строке;
- **memchr(const char*, int, size_t)** - ищет первое вхождение указанного символа в строке определенного размера;
- **strpbrk(const char*, const char*)** - ищет первое вхождение одного из символов одной строки в другой;

```
char str[] = "abcdefghf";  
char *p=strchr(str, 'c');  
if (p) *p = 'C', puts(str);  
p=(char*)memchr(str, 'C', 2);  
if (p) *p = '2', puts(str);  
p=strpbrk(str, "ef");  
cout << p - str;
```

Функции поиска и замены

- **strstr(const char*, const char*)** - поиск первого вхождения подстроки;
- **memset(void *, int, size_t)**; - заменяет первые несколько символов строки на указанный символ

```
char str[] = "abcdefghf";  
char* p = strstr(str, "ef");  
if (p) cout << p - str;  
memset(str, '!', 3);  
puts(str);
```

Сравнение строк

- **strcmp(const char*, const char*)** - сравнивает две строки:
 - 0 - равны строки,
 - отрицательное значение => первая строка < второй,
 - положительное значение => первая строка > второй;
- **strncmp(const char*, const char*, size_t)** - сравнивает две строки определенного размера
- **_stricmp(const char*, const char*)** - сравнивает две строки без учета регистра

```
char str[] = "abcdefghf", str1[] = "abCDefghf";
```

```
int i = _stricmp(str, str1);
```

```
if (!i) cout << "="; else if (i > 0) cout << ">"; else cout << "<";
```

Типизированный указатель `char*`

Пример объявления указателя:

```
char* char* strPM1 = "Hello, World!";  
char* strPM2;
```

Типы `char[]` и `char*` допускают взаимные преобразования.

Пример процедуры копирования строк в стиле C++:

```
void mycopy(char* p, const char* q)  
{  
    while(*p++ = *q++);  
}
```

Эта процедура копирует содержимое строки `q` в строку `p`.

Класс `string`

- обеспечивает работу со строками переменной длины и поддерживает многие полезные операции над строками

- **`#include <string>`**

- `using namespace std;`

```
string s = "abcdef";
```

```
string str("123456", 2,2);
```

```
cout << str<<endl;
```

```
string t;
```

```
t.assign(s.begin(), s.end()-3);
```

```
cout << t << endl;
```

```
t += str;
```

```
cout << t << endl;
```

Как устроена строка string?



- Размер выделенного буфера под строку и длина непосредственно строки могут различаться, именно поэтому нужно хранить как размер самой строки, так и размер буфера.
- В общем случае, мы могли бы иметь объект `string`, состоящий только из одного указателя, условившись всегда хранить нуль-символ (какой-бы он ни был для того или иного типа символа) в конце строки, а также условившись, что размер буфера всегда равен размеру строки.
- Это сделало бы наш объект более лёгким, но каждый раз, когда нам нужно было бы узнать размер строки он бы вычислялся заново.

Основные возможности, которыми обладает класс **string**:

- инициализация массивом символов (строкой встроенного типа) или другим объектом типа `string`. С-строка не обладает второй возможностью;
- копирование одной строки в другую.
- доступ к отдельным символам строки для чтения и записи. Во встроенном массиве для этого применяется операция взятия индекса или косвенная адресация с помощью указателя;
- сравнение двух строк на равенство.

- **конкатенация** (сцепление) двух строк, дающая результат либо как третью строку, либо вместо одной из исходных;
- встроенные средства определения длины строки (функции-члены класса `size()` и `length()`). Узнать длину строки встроенного типа можно только вычислением с помощью функции `strlen()`.
- возможность узнать, пуста ли строка.

Ввод строки string (потокковый)

```
cin >> s; // считывание до пробела
```

```
cout << s << endl; // вывод строки
```

```
getline(cin, s, '\n'); // считывание до указанного  
СИМВОЛА
```

```
cout << s << endl;
```

```
getline(cin, t, '\n');
```

```
cout << t << endl;
```

Обращение к методам string

Имя_строки.метод (набор параметров)

Пример:

Вставка символа на заданную позицию: insert();

```
string str = "Look this example!"
```

```
str.insert(5," at ");
```

Методы string

- `c_str()`; - перевод в `char*`
- `length()`; `size()`; - длина строки
- `resize()`; - задает новое количество символов в строке
- `clear()`; - очищает все символы
- `empty()`; - проверка на пустоту
- `at()`; - возвращает ссылку на элемент с данным номером
- `front()`; - возвращает ссылку на первый элемент
- `back()`; - возвращает ссылку на последний элемент
- `substr()`; - возвращает фрагмент строки, начиная с данного заданной длины

Методы string

- `push_back();` - добавление символа в конец
- `append();` - добавление символов в конец
- `pop_back();` - удаление последнего символа
- `erase();` - удаляет символ или фрагмент
- `swap();` - обмен
- `replace();` - заменяет фрагмент строки символом или подстрокой

Методы string

- `find()`; - поиск символа или фрагмента
- `rfind()`; - поиск символа или фрагмента
- `==`, `!=`, `<`, `<=`, `>`, `>=`
- `compare()`;